

Non-Functional Requirements (NFR)

1. Security (Category: Security)

Description:

Security focuses on protecting sensitive user financial data, including portfolios, assets, and investment preferences, from unauthorized access and attacks.

Why it was prioritized:

- Financial applications handle highly sensitive data, making them prime targets for attacks
- Users must trust the system to securely store and process their financial information
- Preventing breaches is critical to maintaining system integrity and user confidence

MERN + Vite Considerations:

- Node.js + Express: Secure API endpoints using middleware for authentication and authorization
- bcrypt: Password hashing before storing credentials in MongoDB
- Cors: prevents unauthorized access from malicious sites
- HTTPS: Ensures encrypted communication between client and server

2. Performance (Category: Performance)

Description:

Performance focuses on minimizing page load times and ensuring fast rendering of the frontend so users can quickly access the AI assistant and their financial data.

Why it was prioritized:

- Users expect web applications to load within 2 seconds or less
- Slow load times negatively impact user experience and engagement
- The application dynamically loads user portfolios and financial article data, which can increase load time if not optimized
- Fast access is especially important for interacting with the AI assistant in real time

MERN + Vite Considerations:

- Vite: Enables fast initial load times through optimized bundling and efficient asset serving
- React: Reduces unnecessary re-renders using component-based architecture
- Node.js + Express: Handles API requests efficiently with asynchronous operations
- MongoDB: Indexed queries improve data retrieval speed

3. Scalability (Category: Scalability)

Description:

Scalability ensures the system can handle increasing numbers of users, assets, and financial articles without degrading performance.

Why it was prioritized:

- The platform is expected to grow in both users and stored financial data
- AI-related processing increases backend workload over time
- Designing for scalability early prevents costly system redesign

MERN + Vite Considerations:

- Express (backend): Stateless REST APIs allow horizontal scaling across multiple servers
- MongoDB: Flexible schema design supports growing and evolving datasets
- React + Vite: Frontend and backend are decoupled, enabling independent scaling
- Modular architecture (routes, controllers, services) supports easy system expansion

Trade-offs:

1. Security vs Usability

Trade-off:

By prioritizing strong security measures such as bcrypt hashing, JWT authentication, and CORS restrictions, the application becomes more complex for users to interact with. Users must go through authentication flows and may face stricter session management, rather than enjoying a frictionless, open experience.

Why this is acceptable:

- Financial data is highly sensitive, making security a non-negotiable priority over convenience
- The minor friction of logging in is a widely accepted norm in financial applications and users expect it
- A single security breach would cause far greater harm to user trust than a slightly less seamless experience

Benefits gained:

Protected user portfolios and financial data from unauthorized access

Reduced risk of breaches and attacks

Long-term user trust and system credibility

2. Performance vs Maintainability

Trade-off:

By prioritizing fast load times and efficient rendering through Vite's optimized bundling and React's component architecture, the codebase introduces tightly optimized code paths and performance-specific patterns that can be harder to read, modify, and extend over time. Less time was invested in thorough documentation and code simplicity in favour of speed.

Why this is acceptable:

- Users expect sub-2-second load times, especially when interacting with a real-time AI assistant and live financial data
- Performance issues are immediately visible to users, while maintainability concerns surface gradually over time
- The modular MERN architecture still provides a reasonable foundation for future developers to work within

Benefits gained:

- Faster page loads and smoother user experience
- Efficient real-time interaction with the AI assistant
- Reduced bounce rates and higher user engagement

4. Performance Testing Report

Method and evidence:

Core Web Vitals were captured using the browser Developer Tools Performance panel (Live metrics / Local metrics), evaluating the production-built React client served via the Vite toolchain and Material UI (MUI) components.

Observed metrics (screenshot):

- Largest Contentful Paint (LCP): 0.51 s — labeled “Good.” The tool identified the LCP element as an MUI Typography heading (selector pattern `h4.MuiTypography-root...`), consistent with the React + MUI frontend.
- Cumulative Layout Shift (CLS): 0.01 — labeled “Good,” with a single reported layout shift cluster, indicating high visual stability during load.
- Interaction to Next Paint (INP): 16 ms — labeled “Good,” recorded for a pointer interaction (e.g., click or tap), indicating very low input latency.

Other DevTools indicators (same session):

The Performance panel also reported console diagnostics (e.g., errors, warnings, and issues counts). Those items should be triaged separately from Core Web Vitals; they do not change the “Good” ratings above but may point to follow-up work in application or third-party code.

Comparison to stated expectations:

The NFR document targets sub-2 second perceived load for key content. An LCP of 0.51 s is well within that goal and within the common “Good” threshold (2.5 s). INP at 16 ms is far below the typical “Good” responsiveness threshold (200 ms), supporting real-time chat and navigation. CLS

at 0.01 is well below the usual “Good” threshold (0.1), reducing disruptive layout changes while users read financial data.

Alignment with implementation:

The stack described earlier (Vite bundling, React component model, Express async handlers, and indexed MongoDB access) is consistent with the measured LCP, INP, and CLS. Fast LCP with an MUI text element as the largest paint target indicates efficient initial render of primary UI chrome and content.

5. Security Measures & Testing

Authentication and session management:

- Local accounts: Passwords are hashed with bcrypt (salt rounds) before persistence; login compares hashes via bcrypt.compare (server/services/account/accountService.js).
- Email verification: Short-lived JWTs sign verification payloads and are verified with jsonwebtoken before updating account state (server/services/email/emailService.js).
- Google OAuth: Authorization-code flow exchanges code for tokens and loads user info from Google’s userinfo endpoint before upserting the account (server/services/account/accountService.js).
- Sessions: express-session stores userId and profile fields after login; logout destroys the session. Session secret is loaded from environment (server/app.js, server/utls/constants.js).

Authorization:

- Protected routes use the checkAuth middleware, which requires req.session.userId or returns 401 (server/middleware/checkAuth.js). Applied to profile routes and all /api/assets routes (server/routes/profile.js, server/routes/assetRoutes.js).
- Chat history and per-session messages tie access to the authenticated user and enforce ownership where applicable (server/controllers/chatController.js, server/services/chat/chatService.js — e.g., session/message ownership checks).

Network and API surface:

- CORS is restricted to CLIENTURL from environment with credentials enabled (server/app.js, server/utls/constants.js). This serves as a client protection mechanism
- Public account and chat POST endpoints are intentionally open for signup/login and guest chat; sensitive portfolio and asset operations remain behind checkAuth.

Data layer and validation:

- MongoDB is accessed through Mongoose models with typed schemas and enums (e.g., Profile, Account), reducing ad-hoc query construction. There is no SQL layer; traditional SQL injection does not apply. Query patterns use model methods and ObjectId validation where implemented (e.g., chat service).
- Account controllers validate presence of required body fields for login/signup (server/controllers/account.js). Profile updates use runValidators on upsert paths (server/services/profile/profileService.js).

- CSV upload uses multer with memory storage for portfolio import (server/routes/assetRoutes.js).

Client-side rendering and XSS:

- User-authored chat content is rendered as plain text via MUI Typography (escaped). Assistant messages use react-markdown with remark-gfm without enabling raw HTML plugins in the reviewed component (client/src/components/chat/messagesList/MessagesList.jsx), which limits script injection compared to unsanitized HTML rendering.

Results of basic security tests (automated smoke):

Command: npm run test:security (Jest, server/ directory). Latest run: 2 test suites, 4 tests, all passed (~2 s). Configuration: jest.config.security.js runs only security smoke files and skips the MongoMemoryServer global setup used by other tests.

SQL injection:

Not applicable. The application persists data with MongoDB and Mongoose, not a relational SQL database, so classic SQL injection against string-concatenated queries does not apply. API access still uses parameterized-style Mongoose queries (e.g., findOne({ email })) rather than raw SQL strings.

NoSQL injection–style payloads (MongoDB operator-shaped fields):

- Unit tests send operator-shaped values in the email field (e.g., { \$gt: "" }, { \$ne: null }) to the account service layer (server/__tests__/unit/securityInjection.smoke.test.js).
- Observed behavior: Mongoose forwards the query shape to findOne; when no document matches, login returns 400 “account does not exist”; when a user is returned, bcrypt password comparison still fails with a wrong password (401).
- Signup duplicate check with an operator-shaped email is passed through to findOne; if a matching user exists, signup is rejected with 400 as expected.
- Conclusion: these smoke tests did not demonstrate authentication bypass via the tested payloads; defense-in-depth recommendation: validate that email and password are strings before persistence and queries.

Cross-site scripting (XSS) prevention:

- A React server-side render test (renderToStaticMarkup) confirms that string children containing <script> and markup are emitted with HTML entities (e.g., <script>), not as live DOM tags (server/__tests__/unit/securityReactEscape.test.js).
- This matches the client’s use of text for user messages and Markdown for assistant output without raw-HTML plugins in the reviewed chat component.

HTTPS:

Transport security is assumed to be enforced by the hosting environment (reverse proxy or platform TLS). The application code does not terminate TLS itself; operators should serve the API and static client over HTTPS in production.

Gaps and recommendations:

- No helmet (security headers), rate limiting, or express-mongo-sanitize middleware appears in server/app.js — consider adding for production hardening.
- Multer is configured without explicit file size or MIME restrictions; define limits to mitigate denial-of-service via large uploads.
- Session cookies: default express-session settings are used; for production HTTPS deployments, configure cookie.secure and appropriate sameSite attributes.
- Ongoing dependency and console-error triage: address DevTools-reported errors/warnings independently of Core Web Vitals.

6. Scalability & Availability Considerations

Architecture:

- The Express API is stateless aside from session state stored in the session mechanism; business logic is organized into routes, controllers, and services, supporting horizontal scale of app servers behind a load balancer when session storage is externalized (e.g., shared store) — not configured in-repo.
- The React (Vite) client is decoupled and can be deployed to static hosting separately from the API.

Data and query performance:

- MongoDB indexes: e.g., compound index on Chunk (source_id, chunk_index) and indexed profile userId (server/models/Chunks.js, server/models/profile.js). Account email is unique by schema.
- Article indexing pipeline runs embedding sequentially to avoid external API rate limits (server/services/article/indexService.js), trading throughput for reliability under provider quotas.

Caching and load balancing:

No in-application HTTP cache, CDN configuration, Redis cache, or load balancer definitions were found in the repository. Scaling reads and static assets would typically add a CDN and, for API hot paths, a cache or read replicas — to be introduced at deployment time.

Deployment strategy and uptime considerations:

The team intends to pursue a fully cloud-based deployment to improve stability and operational consistency (managed infrastructure, databases, and related services as appropriate to the chosen provider).

Uptime considerations:

- Proactive monitoring: use tools such as Nagios, Datadog, or UptimeRobot to check site availability at frequent intervals (e.g., every minute), with alerting when checks fail or latency thresholds are exceeded.

- Data center location and security: prefer hosting providers with robust, geographically dispersed data centers to reduce correlated risk from localized environmental events or regional power failures.
- Software and security operations: regularly patch, update, and harden systems; mitigate DDoS and abuse at the network and application layers, since security incidents and attacks are a common cause of unplanned downtime.

Application-level hooks (current codebase):

- Configuration is environment-driven (dotenv) for MongoDB URI, session secret, JWT secret, OAuth credentials, and AI endpoints (server/utils/constants.js).
- A simple health check exists at GET /api/health (server/app.js) for uptime probes.
- No Dockerfile, docker-compose, or CI workflow files were present in the workspace snapshot; concrete cloud resources and runbooks will be finalized as part of the deployment plan.